

Group A

Assignment No: 1

Title of the Assignment: Data Wrangling, I

Perform the following operations using Python on any open source dataset (e.g., data.csv)

Import all the required Python Libraries.

1. Locate open source data from the web (e.g. <https://www.kaggle.com>).
2. Provide a clear description of the data and its source (i.e., URL of the web site).
3. Load the Dataset into the pandas data frame.
4. Data Preprocessing: check for missing values in the data using pandas `isnull()`, `describe()` function to get some initial statistics. Provide variable descriptions. Types of variables etc. Check the dimensions of the data frame.
5. Data Formatting and Data Normalization: Summarize the types of variables by checking the data types (i.e., character, numeric, integer, factor, and logical) of the variables in the data set. If variables are not in the correct data type, apply proper type conversions.
6. Turn categorical variables into quantitative variables in Python.

Introduction to Data Wrangling

Data wrangling is the process of cleaning and unifying messy and complex data sets for easy access and analysis.

With the amount of data and data sources rapidly growing and expanding, it is getting increasingly essential for large amounts of available data to be organized for analysis. This process typically includes manually converting and mapping data from one raw form into another format to allow for more convenient consumption and organization of the data.

The Goals of Data Wrangling

Reveal a "deeper intelligence" by gathering data from multiple sources

Provide accurate, actionable data in the hands of business analysts in a timely matter

Reduce the time spent collecting and organizing unruly data before it can be utilized

Enable data scientists and analysts to focus on the analysis of data, rather than the wrangling

Drive better decision-making skills by senior leaders in an organization

Key Steps to Data Wrangling

Data Acquisition: Identify and obtain access to the data within your sources.

Joining Data: Combine the edited data for further use and analysis.

Data Cleansing: Redesign the data into a usable and functional format and correct/remove any bad data.

Panda functions for Data Formatting and Normalization

The Transforming data stage is about converting the data set into a format that can be analyzed or modelled effectively, and there are several techniques for this process.

a. **Data Formatting:** Ensuring all data formats are correct (e.g. object, text, floating number, integer, etc.) is another part of this initial 'cleaning' process. If you are working with dates in Pandas, they also need to be stored in the exact format to use special date-time functions.

b. **Data normalization:** Mapping all the nominal data values onto a uniform scale (e.g. from 0 to 1) is involved in data normalization. Making the ranges consistent across variables helps with statistical analysis and ensures better comparisons later on..

Algorithm:

Step 1 : Import all the required Python Libraries.

Step 2: Load the weather dataset from Kaggle in dataframe object df

Step 3: Print dataset.

```
df.head()
```

```
df.tail()
```

Step 4.1: Describe function returns the statistical summary of the data frame or series.

```
df.describe()
```

Step 4.2: identify missing data A) convert the "?","blackspace" into NaN(non numerical data) function: replace() The replace () method replaces the specified value with another specified value. The replace() method searches the entire Data Frame and replaces every case of specified value.

```
df.replace(" ", np.nan, inplace = True)
```

```
df.head(9)
```

Step 4.3: Check if missing value are present.

```
missingdata=df.isnull()
```

```
df.isnull().sum()
```

Step 4.4: Deal with missing Data a. Drop data Functions:dropna() b. replace data calculate the mean()

```
df.dropna()
```

Step 4.5: Replace Data Calculate the mean and replace the null value by mean by using fillna() function

```
mean_value = df['temperature'].mean()
```

```
df['temperature'] = df['temperature'].fillna(mean_value)
```

```
mean_v=df['windspeed'].mean()
```

```
new_df = df.fillna({'temperature':0,
```

```
'windspeed':0,
```

```
'event':'no_event'})
```

```
new_df
```

Step 5: Summarize the types of variables by checking the data types

```
df.dtypes
```

```
df[["duration"]]=df[["duration"]].astype("float")
```

```
df.head()
```

Step 7: Turn categorical variables into quantitative variables

```
df3 = df.copy()
```

```
df3 = pd.get_dummies(df3,
```

```
                      columns = ['event'])
```

```
display(df3)
```

Conclusion- In this way we have explored the functions of the python library for Data Preprocessing, Data Wrangling Techniques and How to Handle missing values.

Title of the Assignment 2: Data Wrangling, II

Create an “Academic performance” dataset of students and perform the following operations using Python.

1. Scan all variables for missing values and inconsistencies. If there are missing values and/or inconsistencies, use any of the suitable techniques to deal with them.
2. Scan all numeric variables for outliers. If there are outliers, use any of the suitable techniques to deal with them.
3. Apply data transformations on at least one of the variables. The purpose of this transformation should be one of the following reasons: to change the scale for better understanding of the variable, to convert a non-linear relation into a linear one, or to decrease the skewness and convert the distribution into a normal distribution.

Reason and document your approach properly.

Objective of the Assignment: Students should be able to perform the data wrangling operation using Python on any open source dataset

Prerequisite:

1. Basic of Python Programming
2. Concept of Data Preprocessing, Data Formatting , Data Normalization and Data Cleaning.

Contents for Theory:

1. Creation of Dataset using Microsoft Excel.
2. Identification and Handling of Null Values
3. Identification and Handling of Outliers
4. Data Transformation for the purpose of :
 - a. To change the scale for better understanding
 - b. To decrease the skewness and convert distribution into normal distribution

Theory:

1. Creation of Dataset using Microsoft Excel. The dataset is created in “CSV” format.

- The name of dataset is StudentsPerformance
- The features of the dataset are: Math_Score, Reading_Score, Writing_Score, Placement_Score, Club_Join_Date .
- Number of Instances: 30
- The response variable is: Placement_Offer_Count .
- Range of Values: Math_Score [60-80], Reading_Score[75-,95], ,Writing_Score [60,80], Placement_Score[75-100], Club_Join_Date [2018-2021].
- The response variable is the number of placement offers facilitated to particular students, which is largely depend on Placement_Score To fill the values in the dataset the RANDBETWEEN is used. Returns a random integer number between the numbers you specify

Syntax : RANDBETWEEN(bottom, top) Bottom The smallest integer and Top The largest integer RANDBETWEEN will return.

For better understanding and visualization, 20% impurities are added into each variable to the dataset. Identification and Handling of Null Values Missing Data can occur when no information is provided for one or more items or for a whole unit. Missing Data is a very big problem in real-life scenarios. Missing Data can also refer to as NA (Not Available) values in pandas. In Data Frame sometimes many datasets simply arrive with missing data, either because it exists and was not collected or it never existed. For Example, suppose different users being surveyed may choose not to share their income, some users may choose not to share the address in this way many datasets went missing.s

2.Checking for missing values using notnull ()

In order to check null values in Pandas Dataframe, notnull () function is used. This function return dataframe of Boolean values which are False for NaN values. Deleting null values using dropna () method

In order to drop null values from a dataframe, dropna () function is used. This function drops

Rows/Columns of datasets with Null values in different ways.

1. Dropping rows with at least 1 null value
2. Dropping rows if all values in that row are missing
3. Dropping columns with at least 1 null value.
4. Dropping Rows with at least 1 null value in CSV file

3. Identification and Handling of Outliers

3.1 Identification of Outliers

One of the most important steps as part of data preprocessing is detecting and treating the outliers as they can negatively affect the statistical analysis and the training process of a machine learning algorithm resulting in lower accuracy.

What are Outliers?

We all have heard of the idiom 'odd one out' which means something unusual in comparison to the others in a group. Similarly, an Outlier is an observation in a given dataset that lies far from the rest of the observations. That means an outlier is vastly larger or smaller than the remaining values in the set.

Why do they occur?

An outlier may occur due to the variability in the data, or due to experimental error/human error. They may indicate an experimental error or heavy skewness in the data (heavy-tailed distribution).

What do they affect?

In statistics, we have three measures of central tendency namely Mean, Median, and Mode. They help us describe the data. Mean is the accurate measure to describe the data when we do not have any outliers present. Median is used if there is an outlier in the dataset. Mode is used if there is an outlier AND about $\frac{1}{2}$ or more of the data is the same. 'Mean' is the only measure of central tendency that is affected by the outliers which in turn impacts Standard deviation.

3.2 Detecting Outliers

If our dataset is small, we can detect the outlier by just looking at the dataset. But what if we have a huge dataset, how do we identify the outliers then? We need to use visualization and

mathematical techniques. Below are some of the techniques of detecting outliers

- Boxplots
- Scatterplots
- Z-score
- Inter Quantile Range(IQR)

3.2.1 Detecting outliers using Boxplot:

It captures the summary of the data effectively and efficiently with only a simple box and whiskers. Boxplot summarizes sample data using 25th, 50th, and 75th percentiles. One can just get insights (quartiles, median, and outliers) into the dataset by just looking at its boxplot.

3.2.2 Detecting outliers using Scatterplot:

It is used when you have paired numerical data, or when your dependent variable has multiple values for each reading independent variable, or when trying to determine the relationship between the two variables. In the process of utilizing the scatter plot, one can also use it for outlier detection.

To plot the scatter plot one requires two variables that are somehow related to each other. So here Placement score and Placement count features are used.

3.2.3 Detecting outliers using Z-Score:

Z-Score is also called a standard score. This value/score helps to understand how far is the data point

from the mean. And after setting up a threshold value one can utilize z score values of data points to

define the outliers.

3.2.4 Detecting outliers using Inter Quantile Range(IQR):

IQR (Inter Quartile Range) Inter Quartile Range approach to finding the outliers is the most commonly used and most trusted approach used in the research field.

$IQR = \text{Quartile}_3 - \text{Quartile}_1$

To define the outlier base value is defined above and below datasets normal range namely Upper and Lower bounds, define the upper and the lower bound ($1.5 \times IQR$ value is considered) :

$$\text{upper} = Q_3 + 1.5 \times IQR$$

$$\text{lower} = Q_1 - 1.5 \times IQR$$

In the above formula as according to statistics, the 0.5 scale-up of IQR (new_IQR

= IQR + 0.5*IQR) is taken.

5. Handling of Outliers:

For removing the outlier, one must follow the same process of removing an entry from the dataset using its exact position in the dataset because in all the above methods of detecting the outliers end result is the list of all those data items that satisfy the outlier definition according to the method used.

Below are some of the methods of treating the outliers

- Trimming/removing the outlier
- Quantile based flooring and capping
- Mean/Median imputation

Conclusion: In this way we have explored the functions of the python library for Data Identifying and handling the outliers. Data Transformations Techniques are explored with the purpose of creating the new variable and reducing the skewness from datasets.

Group A

Assignment No 3

Title of the Assignment:

Descriptive Statistics - Measures of Central Tendency and variability

Perform the following operations on any open source dataset (e.g., data.csv)

1. Provide summary statistics (mean, median, minimum, maximum, standard deviation) for a dataset (age, income etc.) with numeric variables grouped by one of the qualitative (categorical) variables. For example, if your categorical variable is age groups and quantitative variable is income, then provide summary statistics of income grouped by the age groups. Create a list that contains a numeric value for each response to the categorical variable.
2. Write a Python program to display some basic statistical details like percentile, mean, standard deviation etc. of the species of „Iris-setosa“, „Iris-versicolor“ and „Iris- versicolor“ of iris.csv dataset.

Provide the codes with outputs and explain everything that you do in this step.

Outcome of the Assignment: Students should be able to perform the Statistical operations using Python on any open source dataset.

Prerequisite:

1. Basic of Python Programming
2. Concept of statistics such as mean, median, minimum, maximum, standard deviation etc.

1. Summary statistics:

- What is Statistics?

Statistics is the science of collecting data and analyzing them to infer proportions (sample) that are representative of the population. In other words, statistics is interpreting data in order to make predictions for the population.

Branches of Statistics: There are two branches of Statistics.

DESCRIPTIVE STATISTICS: Descriptive Statistics is a statistics or a measure that describes the data.

INFERENTIAL STATISTICS: Using a random sample of data taken from a population to describe and make inferences about the population is called Inferential Statistics.

Descriptive Statistics:

Descriptive Statistics is summarizing the data at hand through certain numbers like mean, median etc. so as to make the understanding of the data easier. It does not involve any generalization or inference beyond what is available. This means that the descriptive statistics are just the representation of the data (sample) available and not based on any theory of probability.

Commonly Used Measures

1. Measures of Central Tendency
2. Measures of Dispersion (or Variability)

• Measures of Central Tendency

A Measure of Central Tendency is a one number summary of the data that typically describes the centre of the data.

This one number summary is of three types.

a. Mean :

Mean is defined as the ratio of the sum of all the observations in the data to the total number of observations. This is also known as Average. Thus mean is a number around which the entire data set is spread. Consider the following data points. 17, 16, 21, 18, 15, 17, 21, 19, 11, 23

$$\text{Mean} = \frac{17 + 16 + 21 + 18 + 15 + 17 + 21 + 19 + 11 + 23}{10} = \frac{178}{10} = 17.8$$

b. Median :

Median is the point which divides the entire data into two equal halves. One-half of the data is less than the median, and the other half is greater than the same. Median is calculated by first arranging the data in either ascending or descending order. ○ If the number of observations is odd, the median is given by the middle observation in the sorted form. ○ If the number of observations are even, median is given by the mean of the two middle observations in the sorted form. An important point to note is that the order of the data (ascending or descending) does not affect the median.

To calculate Median, let's arrange the data in ascending order. 11, 15, 16, 17, 17, 18, 19, 21, 21, 23

Since the number of observations is even (10), median is given by the average of the two middle observations (5th and 6th here).

$$\text{Median} = \frac{5^{\text{th}} \text{ Obs} + 6^{\text{th}} \text{ Obs}}{2} = \frac{17 + 18}{2} = 17.5$$

c. **Mode :**

Mode is the number which has the maximum frequency in the entire data set, or in other words, mode is the number that appears the maximum number of times. A data can have one or more than one mode.

- If there is only one number that appears the maximum number of times, the data has one mode, and is called Uni-modal.
- If there are two numbers that appear the maximum number of times, the data has two modes, and is called Bi-modal.
- If there are more than two numbers that appear the maximum number of times, the data has more than two modes, and is called Multi-modal.

Consider the following data points. 17, 16, 21, 18, 15, 17, 21, 19, 11, 23

Mode is given by the number that occurs the maximum number of times. Here, 17 and 21 both occur twice. Hence, this is a Bimodal data and the modes are 17 and 21.

• **Measures of Dispersion (or Variability)**

Measures of Dispersion describes the spread of the data around the central value (or the Measures of Central Tendency)

1. Absolute Deviation from Mean — The Absolute Deviation from Mean, also called Mean Absolute Deviation (MAD), describes the variation in the data set, in the sense that it tells the average absolute distance of each data point in the set. It is calculated as

$$\text{Mean Absolute Deviation} = \frac{1}{N} \sum_{i=1}^N |X_i - \bar{X}|$$

2. Variance — Variance measures how far are data points spread out from the mean. A high variance indicates that data points are spread widely and a small variance indicates that the data points are closer to the mean of the data set. It is calculated as

$$\text{Variance} = \frac{1}{N} \sum_{i=1}^N (X_i - \bar{X})^2$$

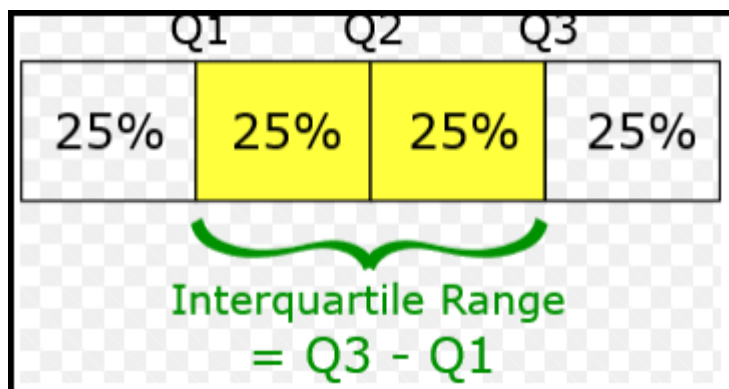
3. Standard Deviation — The square root of Variance is called the Standard Deviation. It is calculated as

$$\text{Std Deviation} = \sqrt{\text{Variance}} = \sqrt{\frac{1}{N} \sum_{i=1}^N (X_i - \bar{X})^2}$$

4. Range — Range is the difference between the Maximum value and the Minimum value in the data set. It is given as

$$\text{Range} = \text{Maximum} - \text{Minimum}$$

5. Quartiles — Quartiles are the points in the data set that divides the data set into four equal parts. Q1, Q2 and Q3 are the first, second and third quartile of the data set.
- 25% of the data points lie below Q1 and 75% lie above it.
 - 50% of the data points lie below Q2 and 50% lie above it. Q2 is nothing but Median.
 - 75% of the data points lie below Q3 and 25% lie above it.

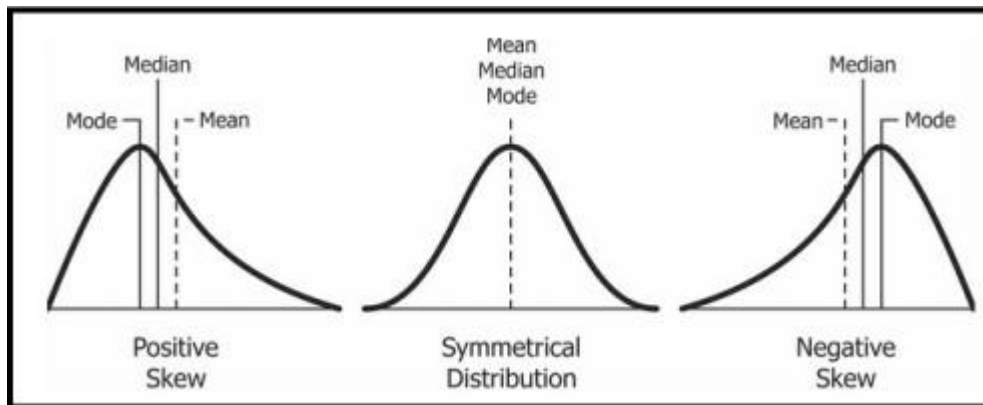


6. Skewness — The measure of asymmetry in a probability distribution is defined by Skewness. It can either be positive, negative or undefined.

$$\text{Skewness} = \frac{3 (\text{Mean} - \text{Median})}{\text{Std Deviation}}$$

Positive Skew — This is the case when the tail on the right side of the curve is bigger than that on the left side. For these distributions, mean is greater than the mode.

Negative Skew — This is the case when the tail on the left side of the curve is bigger than that on the right side. For these distributions, mean is smaller than the mode. The most commonly used method of calculating Skewness is If the skewness is zero, the distribution is symmetrical. If it is negative, the distribution is Negatively Skewed and if it is positive, it is Positively Skewed.



Python Code:

1. Mean

To find mean of all columns

Syntax: `df.mean()`

To find mean of specific column

Syntax: `df.loc[:, 'Age'].mean()`

To find mean row wise

Syntax: `df.mean(axis=1)[0:4]`

2. Median

To find median of all columns

Syntax: `df.median()`

To find median of specific column

Syntax: `df.loc[:, 'Age'].median()`

To find median row wise

Syntax: `df.median(axis=1)[0:4]`

3. Mode

To find mode of all columns

Syntax: `df.mode()`

To find the mode of a specific column.

Syntax: `df.loc[:, 'Age'].mode()`

4. Minimum

To find minimum of all columns

Syntax: `df.min()`

To find minimum of Specific column

Syntax: `df.loc[:, 'Age'].min(skipna = False)`

5. Maximum

To find Maximum of all columns

Syntax: `df.max()`

6. Standard Deviation

To find Standard Deviation of all columns

Syntax: `df.std()`

To find Standard Deviation of specific column

Syntax: `df.loc[:, 'Age'].std()`

2. Types of Variables:

A variable is a characteristic that can be measured and that can assume different values. Height, age, income, province or country of birth, grades obtained at school and type of housing are all examples of variables.

Variables may be classified into two main categories:

- Categorical and
- Numeric.

Each category is then classified in two subcategories: nominal or ordinal for categorical variables, discrete or continuous for numeric variables.

Categorical variables:

A categorical variable (also called qualitative variable) refers to a characteristic that can't be quantifiable.

Categorical variables can be either nominal or ordinal.

- **Nominal Variable** A nominal variable is one that describes a name, label or category without natural order. In the given table, the variable “mode of transportation for travel to work” is also nominal.

Method of travel to work for Canadians	
Mode of transportation for travel to work	Number of people
Car, truck, van as driver	9,929,470
Car, truck, van as passenger	923,975
Public transit	1,406,585
Walked	881,085
Bicycle	162,910
Other methods	146,835

- **Ordinal Variable** An ordinal variable is a variable whose values are defined by an order relation between the different categories. In the following table, the variable “behaviour” is ordinal because the category “Excellent” is better than the category “Very good,” which is better than the category “Good,” etc. There is some natural ordering, but it is limited since we do not know by how much “Excellent” behaviour is better than “Very good” behaviour.

Student behaviour ranking	
Behaviour	Number of students
Excellent	5
Very good	12
Good	10
Bad	2
Very bad	1

Numerical Variables :

A numeric variable (also called quantitative variable) is a quantifiable characteristic whose values are numbers (except numbers which are codes standing up for categories). Numeric variables may be either continuous or discrete.

- **Continuous variables**

A variable is said to be continuous if it can assume an infinite number of real values within a given interval. For instance, consider the height of a student. The height can’t take any values. It can’t be negative and it can’t be higher than three metres. But

between 0 and 3, the number of possible values is theoretically infinite. A student may be 1.6321748755 ... metres tall.

- Discrete variables

As opposed to a continuous variable, a discrete variable can assume only a finite number of real values within a given interval. An example of a discrete variable would be the score given by a judge to a gymnast in competition: the range is 0 to 10 and the score is always given to one decimal (e.g. a score of 8.5)

3. Summary statistics of income grouped by the age groups

Problem Statement: For example, if your categorical variable is age groups and quantitative variable is income, then provide summary statistics of income grouped by the age groups.

Create a list that contains a numeric value for each response to the categorical variable.

Categorical Variable: Genre Quantitative Variable:

Age

Syntax: `df.groupby(['Genre'])['Age'].mean()`

```
Genre
Female    38.098214
Male      39.806818
Name: Age, dtype: float64
```

Conclusion- In this way students will be gain the skills to effectively perform statistical operations using Python on open-source datasets.

Group A

Assignment No: 4

Title of the Assignment: Create a Linear Regression Model using Python/R to predict home prices using Boston Housing Dataset (<https://www.kaggle.com/c/boston-housing>).

The Boston Housing dataset contains information about various houses in Boston through different parameters. There are 506 samples and 14 feature variables in this dataset.

The objective is to predict the value of prices of the house using the given features.

Outcome of the Assignment: Students should be able to data analysis using liner regression using Python for any open source dataset

Prerequisite:

1. Basic of Python Programming
 2. Concept of Regression.
-

Contents for Theory:

1. Linear Regression: Univariate and Multivariate
2. Least Square Method for Linear Regression
3. Measuring Performance of Linear Regression
4. Example of Linear Regression
5. Training data set and Testing data set

1. **Linear Regression:** It is a machine learning algorithm based on supervised learning. It targets prediction values on the basis of independent variables.

- It is preferred to find out the relationship between forecasting and variables.
- A linear relationship between a dependent variable (X) is continuous; while independent variable(Y) relationship may be continuous or discrete. A linear relationship should be available in between predictor and target variable so known as Linear Regression.
- Linear regression is popular because the cost function is Mean Squared Error (MSE) which is equal to the average squared difference between an observation's actual and predicted values.
- It is shown as an equation of line like :

$$Y = m \cdot X + b + e$$

Where : b is intercepted, m is slope of the line and e is error term.

This equation can be used to predict the value of target variable Y based on given predictor variable(s) X, as shown in Fig. 1.

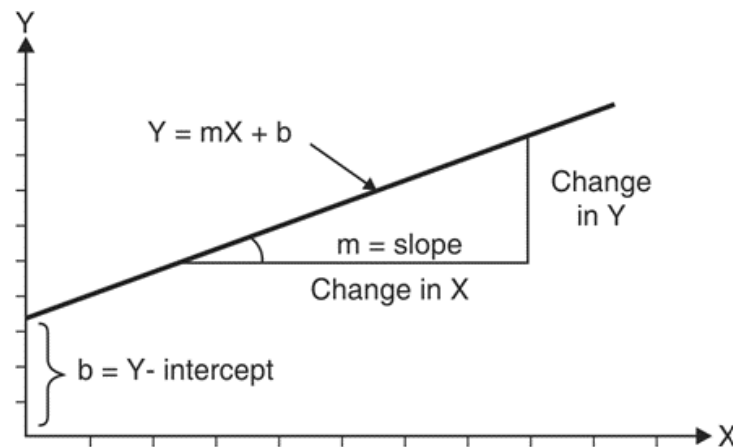
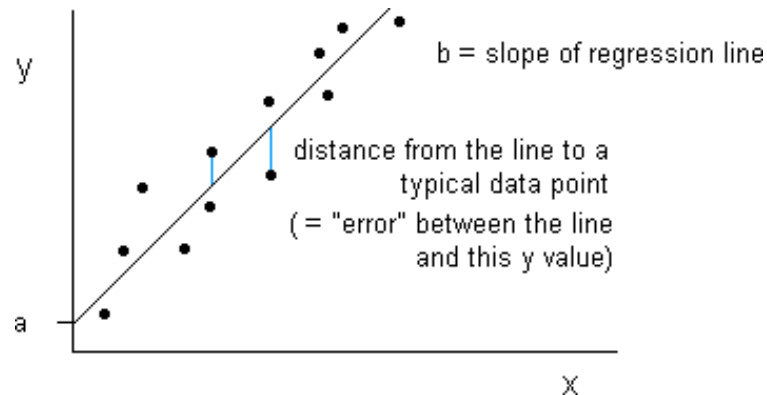


Fig. 1: geometry of linear regression

- Fig. 2 shown below is about the relation between weight (in Kg) and height (in cm), a linear relation. It is an approach of studying in a statistical manner to summarise and learn the relationships among continuous (quantitative) variables.
- Here a variable, denoted by 'x' is considered as the predictor, explanatory, or independent variable.

- Another variable, denoted 'y', is considered as the response, outcome, or dependent variable. While "predictor" and "response" used to refer to these variables.
- Simple linear regression technique concerned with the study of only one predictor variable.

Fig.2 : Relation between weight (in Kg) and height (in cm)



MultiVariate Regression :It concerns the study of two or more predictor variables. Usually a transformation of the original features into polynomial features from a given degree is preferred and further Linear Regression is applied on it.

- A simple linear model $Y = a + bX$ in original feature will be transformed into polynomial feature is transformed and further a linear regression applied to it and it will be something like

$$Y = a + bX + cX^2$$

- If a high degree value is used in transformation the curve becomes over-fitted as it captures the noise from data as well.

2. Least Square Method for Linear Regression

- Linear Regression involves establishing linear relationships between dependent and independent variables. Such a relationship is portrayed in the form of an equation also known as the linear model.

- A simple linear model is the one which involves only one dependent and one independent variable. Regression Models are usually denoted in Matrix Notations.
- However, for a simple univariate linear model, it can be denoted by the regression equation

$$\hat{y} = \beta_0 + \beta_1 x \quad (1)$$

where $\hat{y}$ is the dependent or the response variable

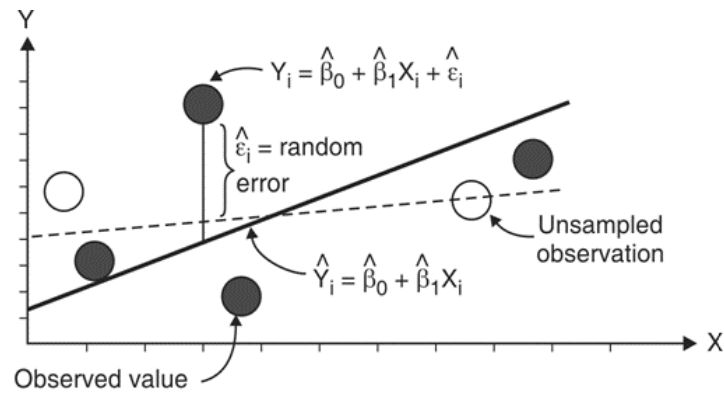
x is the independent or the input variable

β_0 is the value of y when $x=0$ or the y intercept

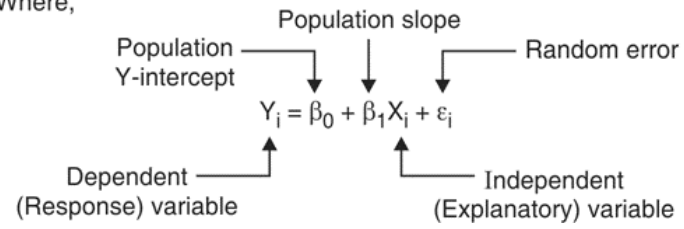
β_1 is the value of slope of the line ϵ is the error or the noise

- This linear equation represents a line also known as the ‘regression line’. The least square estimation technique is one of the basic techniques used to guess the values of the parameters and based on a sample set.
- This technique estimates parameters β_0 and β_1 and by trying to minimise the square of errors at all the points in the sample set. The error is the deviation of the actual sample data point from the regression line. The technique can be represented by the equation.

$$\min \sum_{i=0}^n (\hat{y} - y)^2 \quad (2)$$



Where,



Using differential calculus on equation 1 we can find the values of β_0 and β_1 such that the sum of squares (that is equation 2) is minimum.

$$\beta_1 = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2} \quad (3)$$

$$\beta_0 = \bar{y} - \beta_1 \bar{x} \quad (4)$$

Once the Linear Model is estimated using equations (3) and (4), we can estimate the value of the dependent variable in the given range only. Going outside the range is called extrapolation which is inaccurate if simple regression techniques are used.

3. Measuring Performance of Linear Regression

Mean Square Error:

The Mean squared error (MSE) represents the error of the estimator or predictive model created based on the given set of observations in the sample. Two or more regression models created using a given sample data can be compared based on their MSE. The lesser the MSE, the better the regression model is. When the linear regression model is trained using a given set of obser

ervations, the model with the least mean sum of squares error (MSE) is selected as the best model. The Python or R packages select the best-fit model as the model with the lowest MSE or lowest RMSE when training the linear regression models.

Mathematically, the MSE can be calculated as the average sum of the squared difference between the actual value and the predicted or estimated value represented by the regression model (line or plane).

$$MSE = \frac{1}{n} \sum \underbrace{\left(y - \hat{y} \right)^2}_{\substack{\text{The square of the difference} \\ \text{between actual and} \\ \text{predicted}}}$$


An MSE of zero (0) represents the fact that the predictor is a perfect predictor.

RMSE:

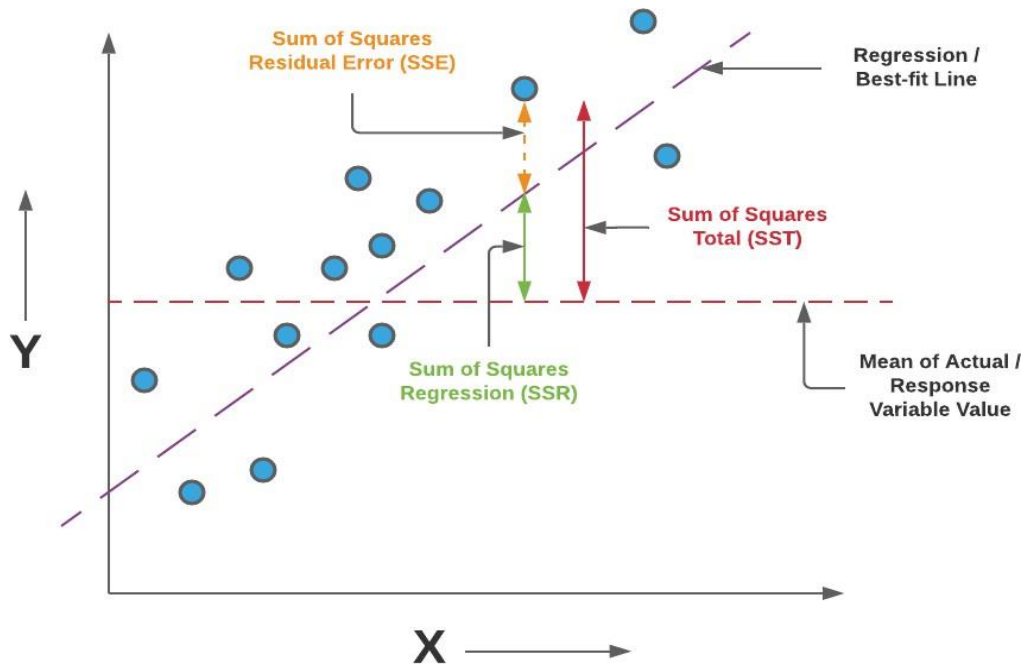
Root Mean Squared Error method that basically calculates the least-squares error and takes a root of the summed values.

Mathematically speaking, Root Mean Squared Error is the square root of the sum of all errors divided by the total number of values. This is the formula to calculate RMSE

$$\text{RMSE} = \sqrt{\sum_{i=1}^n \frac{1}{n} (\hat{y}_i - y_i)^2}$$

RMSE - Least Squares Regression Method - Edureka

R-Squared :



R-Squared is the ratio of the sum of squares regression (SSR) and the sum of squares total (SST).

SST : total sum of squares (SST), regression sum of squares (SSR), Sum of square of errors (SSE) are all showing the variation with different measures.

$$SST = \sum_{i=1} (y_i - \bar{y})^2$$

$$SSR = \sum_{i=1}^n (\hat{y}_i - \bar{y})^2$$

$$SSE = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$R^2 = \frac{SSR}{SST} = \frac{\sum (\hat{y}_i - \bar{y})^2}{\sum (y_i - \bar{y})^2}$$

A value of R-squared closer to 1 would mean that the regression model covers most part of the variance of the values of the response variable and can be termed as a good model.

One can alternatively use MSE or R-Squared based on what is appropriate and the need of the hour. However, the disadvantage of using MSE rather than R-squared is that it will be difficult to gauge the performance of the model using MSE as the value of MSE can vary from 0 to any larger number. However, in the case of R-squared, the value is bounded between 0 and 1.

4. Example of Linear Regression

Consider following data for 5 students.

Each X_i ($i = 1$ to 5) represents the score of i th student in standard X and corresponding Y_i ($i = 1$ to 5) represents the score of i th student in standard XII.

- Linear regression equation best predicts standard XIIth score
- Interpretation for the equation of Linear Regression
- If a student's score is 80 in std X, then what is his expected score in XII standard?

Student	Score in X standard (X_i)	Score in XII standard (Y_i)
1	95	85
2	85	95
3	80	70
4	70	65
5	60	70

x	y	$x - \bar{x}$	$y - \bar{y}$	$(x - \bar{x})^2$	$(x - \bar{x})(y - \bar{y})$
95	85	17	8	289	136
85	95	7	18	49	126
80	70	2	-7	4	-14
70	65	-8	-12	64	96
60	70	-18	-7	324	126
$\bar{x} = 78$	$\bar{y} = 77$			$\Sigma (x - \bar{x})^2 = 730$	$\Sigma (x - \bar{x})(y - \bar{y}) = 470$

(i) linear regression equation that best predicts standard XIIth score

$$\hat{y} = \beta_0 + \beta_1 x$$

$$\beta_1 = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}$$

$$\beta_1 = 470/730 = 0.644$$

$$\beta_0 = \bar{y} - \beta_1 \bar{x}$$

$$\beta_0 = 77 - (0.644 * 78) = 26.768$$

$$\hat{y} = 26.76 + 0.644 x$$

(ii) **Interpretation of the regression line.**

Interpretation 1

For an increase in value of x by 0.644 units there is an increase in value of y in one unit.

Interpretation 2

Even if x = 0 value of independent variable, it is expected that value of y is 26.768

Score in XII standard (Y_i) is 0.644 units depending on Score in X standard (X_i) but other factors will also contribute to the result of XII standard by 26.768 .

(iii) **If a student's score is 65 in std X, then his expected score in XII standard is 78.288**

For $x = 80$ the y value will be

$$\hat{y} = 26.76 + 0.644 * 65 = 68.38$$

5. Training data set and Testing data set

- Machine Learning algorithm has two phases
1. Training and 2. Testing.
- The input of the training phase is training data, which is passed to any machine learning algorithm and machine learning model is generated as output of the training phase.
- The input of the testing phase is test data, which is passed to the machine learning model and prediction is done to observe the correctness of mode.

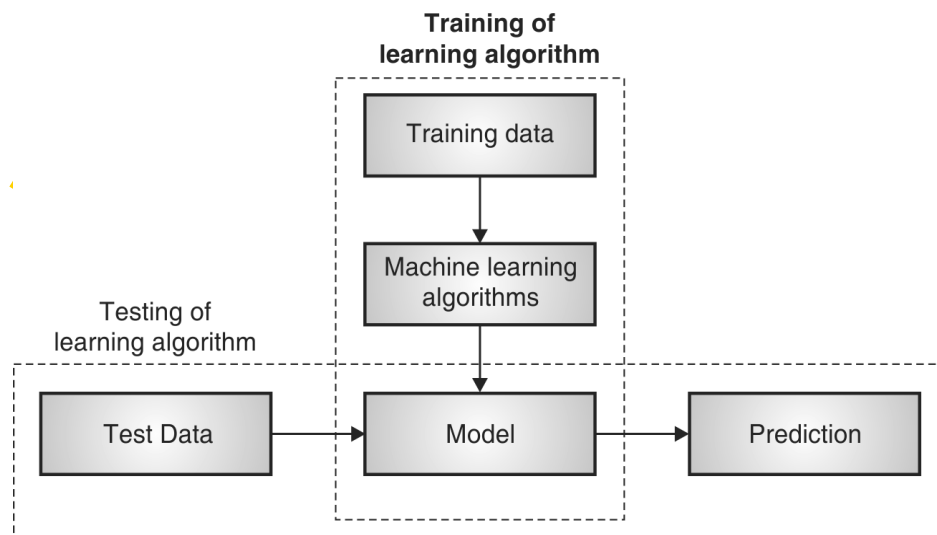


Fig. 1.3.1 : Training and Testing Phase in Machine Learning

(a) Training Phase

- Training dataset is provided as input to this phase.
- Training dataset is a dataset having attributes and class labels and used for training Machine Learning algorithms to prepare models.

- Machines can learn when they observe enough relevant data. Using this one can model algorithms to find relationships, detect patterns, understand complex problems and make decisions.
- Training error is the error that occurs by applying the model to the same data from which the model is trained.
- In a simple way the actual output of training data and predicted output of the model does not match the training error E_{in} is said to have occurred.
- Training error is much easier to compute.

(b) Testing Phase

- Testing dataset is provided as input to this phase.
- Test dataset is a dataset for which class label is unknown. It is tested using model
- A test dataset used for assessment of the finally chosen model.
- Training and Testing dataset are completely different.
- Testing error is the error that occurs by assessing the model by providing the unknown data to the model.
- In a simple way the actual output of testing data and predicted output of the model does not match the testing error E_{out} is said to have occurred.
- E_{out} is generally observed larger than E_{in} .

(c) Generalization

- Generalization is the prediction of the future based on the past system.
- It needs to generalize beyond the training data to some future data that it might not have seen yet.
- The ultimate aim of the machine learning model is to minimize the generalization error.
- The generalization error is essentially the average error for data the model has never seen.
- In general, the dataset is divided into two partition training and test sets.
- The fit method is called on the training set to build the model.
- This fit method is applied to the model on the test set to estimate the target value and evaluate the model's performance.
- The reason the data is divided into training and test sets is to use the test set to estimate how well the model trained on the training data and how well it would perform on the unseen data.

Algorithm (Synthesis Dataset):**Step 1: Import libraries and create alias for Pandas, Numpy and Matplotlib**

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

Step 2: Create a Dataframe with Dependent Variable(x) and independent variable y.

```
x=np.array([95,85,80,70,60])
y=np.array([85,95,70,65,70])
```

Step 3 : Create Linear Regression Model using Polyfit Function:

```
model= np.polyfit(x, y, 1)
```

Step 4: Observe the coefficients of the model.

```
model
```

Output:

```
array([ 0.64383562, 26.78082192])
```

Step 5: Predict the Y value for X and observe the output.

```
predict = np.poly1d(model)
predict(65)
```

Output:

```
68.63
```

Step 6: Predict the y_pred for all values of x.

```
y_pred= predict(x)
y_pred
```

Output:

```
array([81.50684932, 87.94520548, 71.84931507, 68.63013699, 71.84931507])
```

Step 7: Evaluate the performance of Model (R-Square)

R squared calculation is not implemented in numpy... so that one should be borrowed from sklearn.

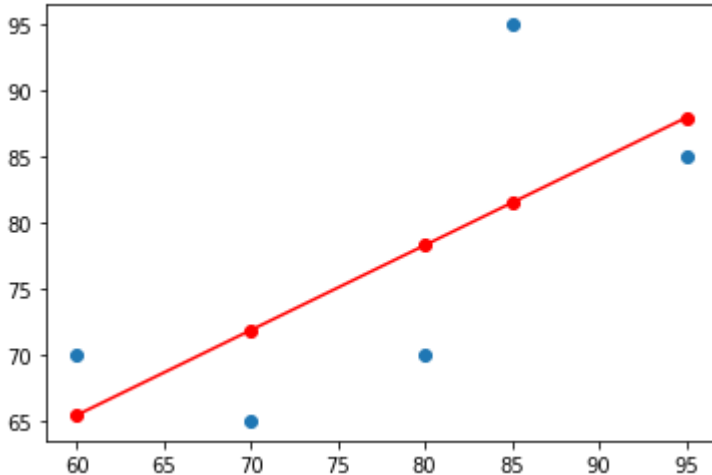
```
from sklearn.metrics import r2_score
r2_score(y, y_pred)
```

Output:

```
0.4803218090889323
```

Step 8: Plotting the linear regression model

```
y_line = model[1] + model[0]* x
plt.plot(x, y_line, c = 'r')
plt.scatter(x, y_pred)
plt.scatter(x,y,c='r')
```

Output:**Algorithm (Boston Dataset):****Step 1: Import libraries and create alias for Pandas, Numpy and Matplotlib**

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

Step 2: Import the Boston Housing dataset

```
from sklearn.datasets import load_boston
boston = load_boston()
```

Step 3: Initialize the data frame

```
data = pd.DataFrame(boston.data)
```

Step 4: Add the feature names to the dataframe

```
data.columns = boston.feature_names
data.head()
```

Step 5: Adding target variable to dataframe

```
data['PRICE'] = boston.target
```

Step 6: Perform Data Preprocessing(Check for missing values)

```
data.isnull().sum()
```

Step 7: Split dependent variable and independent variables

```
x = data.drop(['PRICE'], axis = 1)
y = data['PRICE']
```

Step 8: splitting data to training and testing dataset.

```
from sklearn.model_selection import train_test_split
xtrain, xtest, ytrain, ytest =
train_test_split(x, y, test_size=0.2, random_state = 0)
```

Step 9: Use linear regression(Train the Machine) to Create Model

```
import sklearn
from sklearn.linear_model import LinearRegression
lm = LinearRegression()
model=lm.fit(xtrain, ytrain)
```

Step 10: Predict the y_pred for all values of train_x and test_x

```
ytrain_pred = lm.predict(xtrain)
ytest_pred = lm.predict(xtest)
```

Step 11: Evaluate the performance of Model for train_y and test_y

```
df=pd.DataFrame(ytrain_pred,ytrain)
df=pd.DataFrame(ytest_pred,ytest)
```

Step 12: Calculate Mean Square Paper for train_y and test_y

```
from sklearn.metrics import mean_squared_error, r2_score
mse = mean_squared_error(ytest, ytest_pred)
print(mse)
mse = mean_squared_error(ytrain_pred,ytrain)
print(mse)
```

Output:

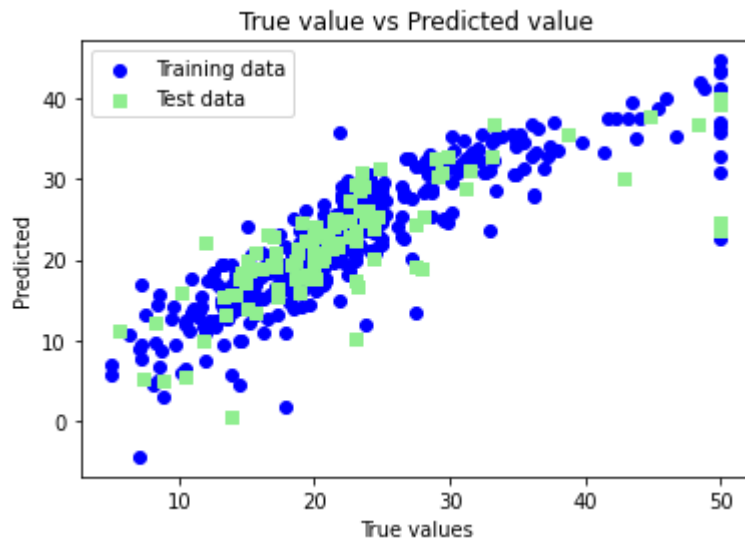
```
33.44897999767638
mse = mean_squared_error(ytest, ytest_pred)
print(mse)
```

Output:

```
19.32647020358573
```

Step 13: Plotting the linear regression model

```
lt.scatter(ytrain ,ytrain_pred,c='blue',marker='o',label='Training data')
plt.scatter(ytest,ytest_pred ,c='lightgreen',marker='s',label='Test data')
plt.xlabel('True values')
plt.ylabel('Predicted')
plt.title("True value vs Predicted value")
plt.legend(loc= 'upper left')
#plt.hlines(y=0,xmin=0,xmax=50)
plt.plot()
plt.show()
```

Conclusion:

In this way we have done data analysis using linear regression for Boston Dataset and predict the price of houses using the features of the Boston Dataset.

Assignment Question:

1) Compute SST, SSE, SSR, MSE, RMSE, R Square for the below example .

Student	Score in X standard (X_i)	Score in XII standard (Y_i)
1	95	85
2	85	95
3	80	70
4	70	65
5	60	70

2) Comment on whether the model is best fit or not based on the calculated values.

3) Write python code to calculate the RSquare for Boston Dataset.

(Consider the linear regression model created in practical session)

Group A

Assignment No: 5

Title of the Assignment:

1. Implement logistic regression using Python/R to perform classification on Social_Network_Ads.csv dataset.
2. Compute Confusion matrix to find TP, FP, TN, FN, Accuracy, Error rate, Precision, Recall on the given dataset..

Objective of the Assignment: Students should be able to data analysis using logistic regression using Python for any open source dataset

Prerequisite:

1. Basic of Python Programming
 2. Concept of Regression.
-

Contents for Theory:

1. Logistic Regression
2. Differentiate between Linear and Logistic Regression
3. Sigmoid Function
4. Types of LogisticRegression
5. Confusion Matrix Evaluation Metrics

1. **Logistic Regression:** Classification techniques are an essential part of machine learning and data mining applications. Approximately 70% of problems in Data Science are classification problems. There are lots of classification problems that are available, but logistic regression is common and is a useful regression method for solving the binary classification problem. Another category of classification is Multinomial classification, which handles the issues where multiple classes are present in the target variable. For example, the IRIS dataset is a very famous example of multi-class classification. Other examples are classifying article/blog/document categories.

Logistic Regression can be used for various classification problems such as spam detection, Diabetes prediction, if a given customer will purchase a particular product or will they churn another competitor, whether the user will click on a given advertisement link or not, and many more examples are in the bucket.

Logistic Regression is one of the most simple and commonly used Machine Learning algorithms for two-class classification. It is easy to implement and can be used as the baseline for any binary classification problem. Its basic fundamental concepts are also constructive in deep learning. Logistic regression describes and estimates the relationship between one dependent binary variable and independent variables.

Logistic regression is a statistical method for predicting binary classes. The outcome or target variable is dichotomous in nature. Dichotomous means there are only two possible classes. For example, it can be used for cancer detection problems. It computes the probability of an event occurring.

It is a special case of linear regression where the target variable is categorical in nature. It uses a log of odds as the dependent variable. Logistic Regression predicts the probability of occurrence of a binary event utilising a logit function.

Linear Regression Equation:

$$y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n$$

Where, y is a dependent variable and x1, x2 ... and Xn are explanatory variables.

Sigmoid Function:

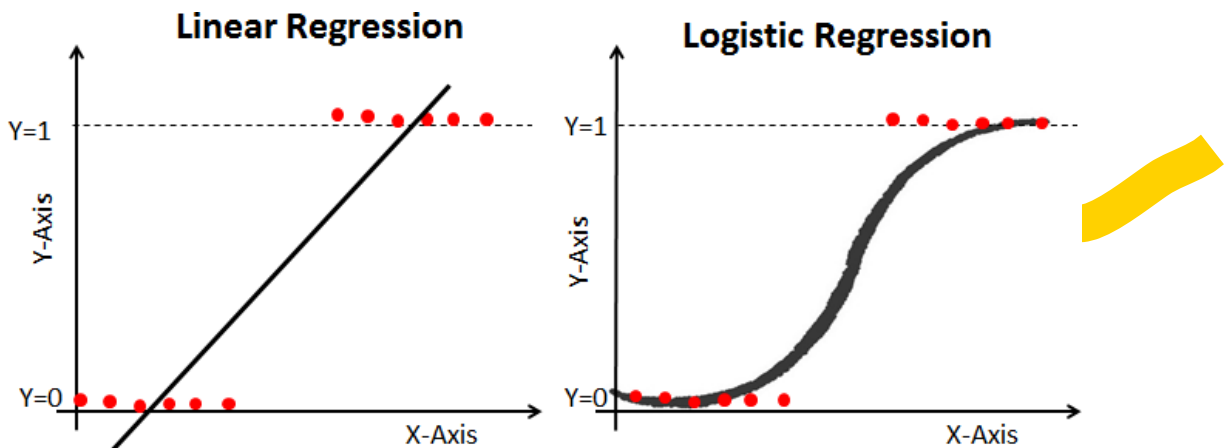
$$p = 1 / (1 + e^{-y})$$

Apply Sigmoid function on linear regression:

$$p = 1 / (1 + e^{-(\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n)})$$

2. Differentiate between Linear and Logistic Regression

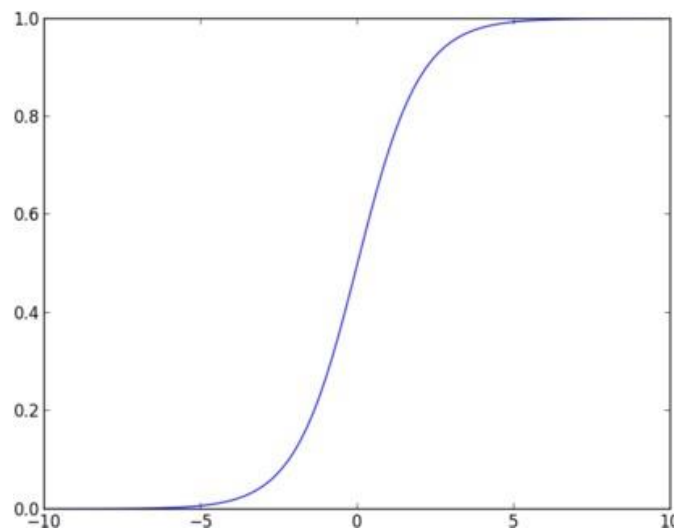
Linear regression gives you a continuous output, but logistic regression provides a constant output. An example of the continuous output is house price and stock price. Example's of the discrete output is predicting whether a patient has cancer or not, predicting whether the customer will churn. Linear regression is estimated using Ordinary Least Squares (OLS) while logistic regression is estimated using Maximum Likelihood Estimation (MLE) approach.



3. Sigmoid Function

The sigmoid function, also called logistic function, gives an 'S' shaped curve that can take any real-valued number and map it into a value between 0 and 1. If the curve goes to positive infinity, y predicted will become 1, and if the curve goes to negative infinity, y predicted will become 0. If the output of the sigmoid function is more than 0.5, we can classify the outcome as 1 or YES, and if it is less than 0.5, we can classify it as 0 or NO. The output cannot be 0.5. For example: If the output is 0.75, we can say in terms of probability as: There is a 75 percent chance that a patient will suffer from cancer.

$$f(x) = \frac{1}{1 + e^{-(x)}}$$



4. Types of Logistic Regression

Binary Logistic Regression: The target variable has only two possible outcomes such as Spam or Not Spam, Cancer or No Cancer.

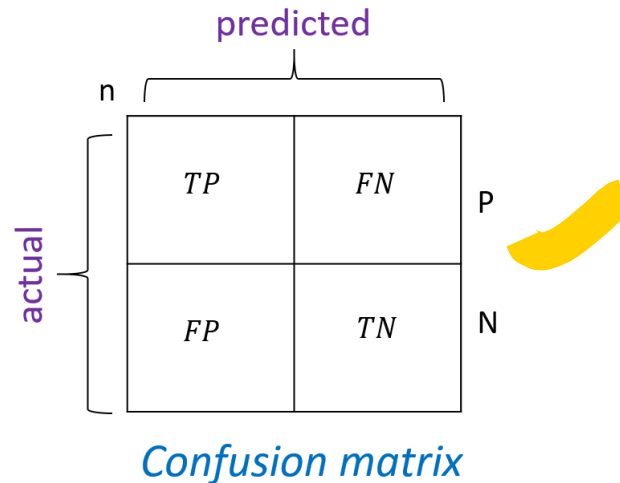
Multinomial Logistic Regression: The target variable has three or more nominal categories such as predicting the type of Wine.

Ordinal Logistic Regression: the target variable has three or more ordinal categories such as restaurant or product rating from 1 to 5.

5. Confusion Matrix Evaluation Metrics

Contingency table or Confusion matrix is often used to measure the performance of classifiers. A confusion matrix contains information about actual and predicted classifications done by a classification system. Performance of such systems is commonly evaluated using the data in the matrix.

The following table shows the confusion matrix for a two class classifier.



Here each row indicates the actual classes recorded in the test data set and the each column indicates the classes as predicted by the classifier.

Numbers on the descending diagonal indicate correct predictions, while the ascending diagonal concerns prediction errors.

Some Important measures derived from confusion matrix are:

- **Number of positive (Pos) :** Total number instances which are labelled as positive in a given dataset.
- **Number of negative (Neg) :** Total number instances which are labelled as negative in a given dataset.
- **Number of True Positive (TP) :** Number of instances which are actually labelled as positive and the predicted class by classifier is also positive.
- **Number of True Negative (TN) :** Number of instances which are actually labelled as negative and the predicted class by classifier is also negative.
- **Number of False Positive (FP) :** Number of instances which are actually labelled as negative and the predicted class by classifier is positive.
- **Number of False Negative (FN):** Number of instances which are actually labelled as positive and the class predicted by the classifier is negative.

- **Accuracy:** Accuracy is calculated as the number of correctly classified instances divided by total number of instances.

The ideal value of accuracy is 1, and the worst is 0. It is also calculated as the sum of true positive and true negative (TP + TN) divided by the total number of instances.

$$acc = \frac{TP+TN}{TP+FP+TN+FN} = \frac{TP+TN}{Pos+Neg}$$

- **Error Rate:** Error Rate is calculated as the number of incorrectly classified instances divided by total number of instances.

The ideal value of accuracy is 0, and the worst is 1. It is also calculated as the sum of false positive and false negative (FP + FN) divided by the total number of instances.

$$err = \frac{FP+FN}{TP+FP+TN+FN} = \frac{FP+FN}{Pos+Neg} \quad \text{Or}$$

$$err = 1 - acc$$

- **Precision:** It is calculated as the number of correctly classified positive instances divided by the total number of instances which are predicted positive. It is also called confidence value. The ideal value is 1, whereas the worst is 0.

$$precision = \frac{TP}{TP+FP}$$

- **Recall:** It is calculated as the number of correctly classified positive instances divided by the total number of positive instances. It is also called recall or sensitivity. The ideal value of sensitivity is 1, whereas the worst is 0.

It is calculated as the number of correctly classified positive instances divided by the total number of positive instances.

$$recall = \frac{TP}{TP+FN}$$

Algorithm (Boston Dataset):**Step 1: Import libraries and create alias for Pandas, Numpy and Matplotlib****Step 2: Import the Social_Media_Adv Dataset****Step 3: Initialize the data frame****Step 4: Perform Data Preprocessing**

- Convert Categorical to Numerical Values if applicable
- Check for Null Value
- Covariance Matrix to select the most promising features
- Divide the dataset into Independent (X) and Dependent (Y) variables.
- Split the dataset into training and testing datasets
- Scale the Features if necessary.

Step 5: Use Logistic regression(Train the Machine) to Create Model

```
# import the class
from sklearn.linear_model import LogisticRegression
# instantiate the model (using the default parameters)
logreg = LogisticRegression()
# fit the model with data
logreg.fit(xtrain,ytrain)
# y_pred=logreg.predict(xtest)
```

Step 6: Predict the y_pred for all values of train_x and test_x**Step 7: Evaluate the performance of Model for train_y and test_y****Step 8: Calculate the required evaluation parameters**

```
from sklearn.metrics import
precision_score, confusion_matrix, accuracy_score, recall_score
cm= confusion_matrix(ytest, y_pred)
```

Conclusion:

In this way we have done data analysis using logistic regression for Social Media Adv. and evaluate the performance of model.

Value Addition: Visualising Confusion Matrix using Heatmap

Assignment Question:

- 1) Consider the binary classification task with two classes positive and negative.

Find out TP, TP, FP, TN, FN, Accuracy, Error rate, Precision, Recall

N = 165	Predicted YES	Predicted NO
Actual YES	TP = 150	FN = 10
Actual NO	FP = 20	TN = 100

- 2) Comment on whether the model is best fit or not based on the calculated values.
- 3) Write python code for the preprocessing mentioned in step 4. and Explain every step in detail.

.

Group A

Assignment No: 6

Title of the Assignment:

1. Implement Simple Naïve Bayes classification algorithm using Python/R on iris.csv dataset.
2. Compute Confusion matrix to find TP, FP, TN, FN, Accuracy, Error rate, Precision, Recall on the given dataset.

Objective of the Assignment: Students should be able to data analysis using Naïve Bayes classification algorithm using Python for any open source dataset

Prerequisite:

1. Basic of Python Programming
 2. Concept of Joint and Marginal Probability.
-

Contents for Theory:

1. Concepts used in Naïve Bayes classifier
 2. Naive Bayes Example
 3. Confusion Matrix Evaluation Metrics
-

1. Concepts used in Naïve Bayes classifier

- Naïve Bayes Classifier can be used for Classification of categorical data.
 - Let there be a 'j' number of classes. $C=\{1,2,...,j\}$
 - Let, input observation is specified by 'P' features. Therefore input observation x is given , $x = \{F1,F2,.....Fp\}$
 - The Naïve Bayes classifier depends on Bayes' rule from probability theory.
- Prior probabilities: Probabilities which are calculated for some event based on no other information are called Prior probabilities.

For example, $P(A)$, $P(B)$, $P(C)$ are prior probabilities because while calculating $P(A)$, occurrences of event B or C are not concerned i.e. no information about occurrence of any other event is used.

Conditional Probabilities:

$$P\left(\frac{A}{B}\right) = \frac{P(A \cap B)}{P(B)} \quad \text{if } P(B) \neq 0 \quad \dots\dots (1)$$

$$P\left(\frac{B}{A}\right) = \frac{P(B \cap A)}{P(A)} \quad \dots\dots\dots (2)$$

From equation (1) and (2) ,

$$P(A \cap B) = P\left(\frac{A}{B}\right) \cdot P(B) = P\left(\frac{B}{A}\right) \cdot P(A)$$

$$\therefore P\left(\frac{A}{B}\right) = \frac{P\left(\frac{B}{A}\right) \cdot P(A)}{P(B)}$$

Is called the Bayes Rule.

2. Example of Naive Bayes

We have a dataset with some features Outlook, Temp, Humidity, and Windy, and the target here is to predict whether a person or team will play tennis or not.

Outlook	Temp	Humidity	Windy	Play
sunny	hot	high	FALSE	no
sunny	hot	high	TRUE	no
overcast	hot	high	FALSE	yes
rainy	mild	high	FALSE	yes
rainy	cool	normal	FALSE	yes
rainy	cool	normal	TRUE	no
overcast	cool	normal	TRUE	yes
sunny	mild	high	FALSE	no
sunny	cool	normal	FALSE	yes
rainy	mild	normal	FALSE	yes
sunny	mild	normal	TRUE	yes
overcast	mild	high	TRUE	yes
overcast	hot	normal	FALSE	yes
rainy	mild	high	TRUE	no

$$X = [\text{Outlook, Temp, Humidity, Windy}]$$

$\underbrace{\hspace{1.5cm}}_{x_1} \quad \underbrace{\hspace{1.5cm}}_{x_2} \quad \underbrace{\hspace{1.5cm}}_{x_3} \quad \underbrace{\hspace{1.5cm}}_{x_4}$

$$C_k = [\text{Yes, No}]$$

$\underbrace{\hspace{1.5cm}}_{C_1} \quad \underbrace{\hspace{1.5cm}}_{C_2}$

Conditional Probability

Here, we are predicting the probability of class1 and class2 based on the given condition. If I try to write the same formula in terms of classes and features, we will get the following equation

$$P(C_k | X) = \frac{P(X | C_k) * P(C_k)}{P(X)}$$



Now we have two classes and four features, so if we write this formula for class C1, it will be something like this.

$$P(C_1 | x_1 \cap x_2 \cap x_3 \cap x_4) = \frac{P(x_1 \cap x_2 \cap x_3 \cap x_4 | C_1) * P(C_1)}{P(x_1 \cap x_2 \cap x_3 \cap x_4)}$$

Here, we replaced Ck with C1 and X with the intersection of X1, X2, X3, X4. You might have a question, It's because we are taking the situation when all these features are present at the same time.

The Naive Bayes algorithm assumes that all the features are independent of each other or in other words all the features are unrelated. With that assumption, we can further simplify the above formula and write it in this form

$$P(C_1 | x_1 \cap x_2 \cap x_3 \cap x_4) = \frac{P(x_1 | C_1) * P(x_2 | C_1) * P(x_3 | C_1) * P(x_4 | C_1) * P(C_1)}{P(x_1) * P(x_2) * P(x_3) * P(x_4)}$$

This is the final equation of the Naive Bayes and we have to calculate the probability of both C1 and C2. For this particular example.

Outlook	Temp	Humidity	Windy	Play
Rainy	Cool	High	True	?

$$P(Yes | X) = P(Rainy | Yes) \times P(Cool | Yes) \times P(High | Yes) \times P(True | Yes) \times P(Yes)$$

$$P(Yes | X) = 2/9 \times 3/9 \times 3/9 \times 3/9 \times 9/14 = 0.00529 \rightarrow 0.2 = \frac{0.00529}{0.02057 + 0.00529}$$

$$P(No | X) = P(Rainy | No) \times P(Cool | No) \times P(High | No) \times P(True | No) \times P(No)$$

$$P(No | X) = 3/5 \times 1/5 \times 4/5 \times 3/5 \times 5/14 = 0.02057 \rightarrow 0.8 = \frac{0.02057}{0.02057 + 0.00529}$$

$P(No | Today) > P(Yes | Today)$ So, the prediction that golf would be played is 'No'.

Algorithm (Iris Dataset):

Step 1: Import libraries and create alias for Pandas, Numpy and Matplotlib

Step 2: Import the Iris dataset by calling URL.

Step 3: Initialize the data frame

Step 4: Perform Data Preprocessing

- Convert Categorical to Numerical Values if applicable
- Check for Null Value

- Divide the dataset into Independent (X) and Dependent (Y) variables.
- Split the dataset into training and testing datasets
- Scale the Features if necessary.

Step 5: Use Naive Bayes algorithm(Train the Machine) to Create Model

```
# import the class
from sklearn.naive_bayes import GaussianNB
gaussian = GaussianNB()
gaussian.fit(X_train, y_train)
```

Step 6: Predict the y_pred for all values of train_x and test_x

```
Y_pred = gaussian.predict(X_test)
```

Step 7: Evaluate the performance of Model for train_y and test_y

```
accuracy = accuracy_score(y_test, Y_pred)
precision = precision_score(y_test, Y_pred, average='micro')
recall = recall_score(y_test, Y_pred, average='micro')
```

Step 8: Calculate the required evaluation parameters

```
from sklearn.metrics import
precision_score, confusion_matrix, accuracy_score, recall_score
cm = confusion_matrix(y_test, Y_pred)
```

Conclusion:

In this way we have done data analysis using Naive Bayes Algorithm for Iris dataset and evaluated the performance of the model.

Value Addition: Visualising Confusion Matrix using Heatmap**Assignment Question:**

- 1) Consider the observation for the car theft scenario having 3 attributes colour, Type and origin.

Example No.	Color	Type	Origin	Stolen?
1	Red	Sports	Domestic	Yes
2	Red	Sports	Domestic	No
3	Red	Sports	Domestic	Yes
4	Yellow	Sports	Domestic	No
5	Yellow	Sports	Imported	Yes
6	Yellow	SUV	Imported	No
7	Yellow	SUV	Imported	Yes
8	Yellow	SUV	Domestic	No
9	Red	SUV	Imported	No
10	Red	Sports	Imported	Yes

Find the probability of car theft having scenarios Red SUV and Domestic.

- 2) Write python code for the preprocessing mentioned in step 4. and Explain every step in detail.

Group A

Assignment No: 7

Title of the Assignment:

1. Extract Sample document and apply following document preprocessing methods:
Tokenization, POS Tagging, stop words removal, Stemming and Lemmatization.
2. Create representation of document by calculating Term Frequency and Inverse Document Frequency.

Objective of the Assignment: Students should be able to perform **Text Analysis** using TF IDF Algorithm

Prerequisite:

1. **Basic of Python Programming**
2. **Basic of English language.**

Contents for Theory:

1. **Basic concepts of Text Analytics**
 2. **Text Analysis Operations using natural language toolkit**
 3. **Text Analysis Model using TF-IDF.**
 4. **Bag of Words (BoW)**
-

1. Basic concepts of Text Analytics

One of the most frequent types of day-to-day conversion is text communication. In our everyday routine, we chat, message, tweet, share status, email, create blogs, and offer opinions and criticism. All of these actions lead to a substantial amount of unstructured text being produced. It is critical to examine huge amounts of data in this sector of the online world and social media to determine people's opinions.

Text mining is also referred to as text analytics. Text mining is a process of exploring sizable textual data and finding patterns. Text Mining processes the text itself, while NLP processes with the underlying metadata. Finding frequency counts of words, length of the sentence, presence/absence of specific words is known as text mining. Natural language processing is one of the components of text mining. NLP helps identify sentiment, finding entities in the sentence, and category of blog/article. Text mining is preprocessed data for text analytics. In Text Analytics, statistical and machine learning algorithms are used to classify information.

2. Text Analysis Operations using natural language toolkit

NLTK(natural language toolkit) is a leading platform for building Python programs to work with human language data. It provides easy-to-use interfaces and lexical resources such as WordNet, along with a suite of text processing libraries for classification, tokenization, stemming, tagging, parsing, and semantic reasoning and many more.

Analysing movie reviews is one of the classic examples to demonstrate a simple NLP Bag-of-words model, on movie reviews.

2.1. Tokenization:

Tokenization is the first step in text analytics. The process of breaking down a text paragraph into smaller chunks such as words or sentences is called Tokenization.

Token is a single entity that is the building blocks for a sentence or paragraph.

- Sentence tokenization : split a paragraph into **list of sentences** using **sent_tokenize()** method

- Word tokenization : split a sentence into **list of words** using **word_tokenize()** method

2.2. Stop words removal

Stopwords considered as noise in the text. Text may contain stop words such as is, am, are, this, a, an, the, etc. In NLTK for removing stopwords, you need to create a list of stopwords and filter out your list of tokens from these words.

2.3. Stemming and Lemmatization

Stemming is a normalization technique where lists of tokenized words are converted into shortened root words to remove redundancy. Stemming is the process of reducing inflected (or sometimes derived) words to their word stem, base or root form.

A computer program that stems word may be called a stemmer.

E.g.

A stemmer reduces the words like fishing, fished, and fisher to the stem fish.

The stem need not be a word, for example the Porter algorithm reduces, argue, argued, argues, arguing, and argus to the stem argu .

Lemmatization in NLTK is the algorithmic process of finding the lemma of a word depending on its meaning and context. Lemmatization usually refers to the morphological analysis of words, which aims to remove inflectional endings. It helps in returning the base or dictionary form of a word known as the lemma.

Eg. Lemma for studies is study

Lemmatization Vs Stemming

Stemming algorithm works by cutting the suffix from the word. In a broader sense cuts either the beginning or end of the word.

On the contrary, Lemmatization is a more powerful operation, and it takes into consideration morphological analysis of the words. It returns the lemma which is the base form of all its inflectional forms. In-depth linguistic knowledge is

required to create dictionaries and look for the proper form of the word. Stemming is a general operation while lemmatization is an intelligent operation where the proper form will be looked in the dictionary. Hence, lemmatization helps in forming better machine learning features.

2.4. POS Tagging

POS (Parts of Speech) tell us about grammatical information of words of the sentence by assigning specific token (Determiner, noun, adjective , adverb , verb, Personal Pronoun etc.) as tag (DT, NN ,JJ, RB, VB, PRP etc) to each words.

Word can have more than one POS depending upon the context where it is used. We can use POS tags as statistical NLP tasks. It distinguishes a sense of word which is very helpful in text realization and infer semantic information from text for sentiment analysis.

3. Text Analysis Model using TF-IDF.

Term frequency–inverse document frequency(TFIDF) , is a numerical statistic that is intended to reflect how important a word is to a document in a collection or corpus.

- **Term Frequency (TF)**

It is a measure of the frequency of a word (w) in a document (d). TF is defined as the ratio of a word's occurrence in a document to the total number of words in a document. The denominator term in the formula is to normalize since all the corpus documents are of different lengths.

$$TF(w, d) = \frac{\text{occurences of } w \text{ in document } d}{\text{total number of words in document } d}$$

Example:

Documents	Text	Total number of words in a document
A	Jupiter is the largest planet	5
B	Mars is the fourth planet from the sun	8

The initial step is to make a vocabulary of unique words and calculate TF for each document. TF will be more for words that frequently appear in a document and less for rare words in a document.

- **Inverse Document Frequency (IDF)**

It is the measure of the importance of a word. Term frequency (TF) does not consider the importance of words. Some words such as 'of', 'and', etc. can be most frequently present but are of little significance. IDF provides weightage to each word based on its frequency in the corpus D.

$$IDF(w, D) = \ln\left(\frac{\text{Total number of documents (N) in corpus D}}{\text{number of documents containing } w}\right)$$

In our example, since we have two documents in the corpus, N=2.

Words	TF (for A)	TF (for B)	IDF
Jupiter	1/5	0	$\ln(2/1) = 0.69$
Is	1/5	1/8	$\ln(2/2) = 0$
The	1/5	2/8	$\ln(2/2) = 0$
largest	1/5	0	$\ln(2/1) = 0.69$
Planet	1/5	1/8	$\ln(2/2) = 0$
Mars	0	1/8	$\ln(2/1) = 0.69$
Fourth	0	1/8	$\ln(2/1) = 0.69$
From	0	1/8	$\ln(2/1) = 0.69$
Sun	0	1/8	$\ln(2/1) = 0.69$

- **Term Frequency — Inverse Document Frequency (TFIDF)**

It is the product of TF and IDF.

TFIDF gives more weightage to the word that is rare in the corpus (all the documents).

TFIDF provides more importance to the word that is more frequent in the document.

$$TFIDF(w, d, D) = TF(w, d) * IDF(w, D)$$

Words	TF (for A)	TF (for B)	IDF	TFIDF (A)	TFIDF (B)
Jupiter	1/5	0	$\ln(2/1) = 0.69$	0.138	0
Is	1/5	1/8	$\ln(2/2) = 0$	0	0
The	1/5	2/8	$\ln(2/2) = 0$	0	0
largest	1/5	0	$\ln(2/1) = 0.69$	0.138	0
Planet	1/5	1/8	$\ln(2/2) = 0$	0.138	0
Mars	0	1/8	$\ln(2/1) = 0.69$	0	0.086
Fourth	0	1/8	$\ln(2/1) = 0.69$	0	0.086
From	0	1/8	$\ln(2/1) = 0.69$	0	0.086
Sun	0	1/8	$\ln(2/1) = 0.69$	0	0.086

After applying TFIDF, text in A and B documents can be represented as a TFIDF vector of dimension equal to the vocabulary words. The value corresponding to each word represents the importance of that word in a particular document.

TFIDF is the product of TF with IDF. Since TF values lie between 0 and 1, not using ***ln*** can result in high IDF for some words, thereby dominating the TFIDF. We don't want that, and therefore, we use ***ln*** so that the IDF should not completely dominate the TFIDF.

- **Disadvantage of TFIDF**

It is unable to capture the semantics. For example, funny and humorous are synonyms, but TFIDF does not capture that. Moreover, TFIDF can be computationally expensive if the vocabulary is vast.

4. **Bag of Words (BoW)**

Machine learning algorithms cannot work with raw text directly. Rather, the text must be converted into vectors of numbers. In natural language processing, a common technique for extracting features from text is to place all of the words that occur in the text in a

bucket. This approach is called a bag of words model or BoW for short. It's referred to as a "bag" of words because any information about the structure of the sentence is lost.

Algorithm for Tokenization, POS Tagging, stop words removal, Stemming and Lemmatization:

Step 1: Download the required packages

```
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('averaged_perceptron_tagger')
```

Step 2: Initialize the text

```
text= "Tokenization is the first step in text analytics. The process
of breaking down a text paragraph into smaller chunks such as words
or sentences is called Tokenization."
```

Step 3: Perform Tokenization

```
#Sentence Tokenization
from nltk.tokenize import sent_tokenize
tokenized_text= sent_tokenize(text)
print(tokenized_text)

#Word Tokenization
from nltk.tokenize import word_tokenize
tokenized_word=word_tokenize(text)
print(tokenized_word)
```

Step 4: Removing Punctuations and Stop Word

```
# print stop words of English
from nltk.corpus import stopwords
stop_words=set(stopwords.words("english"))
print(stop_words)

text= "How to remove stop words with NLTK library in Python?"
text= re.sub('[^a-zA-Z]', ' ',text)
tokens = word_tokenize(text.lower())
filtered_text=[]
for w in tokens:
    if w not in stop_words:
        filtered_text.append(w)
print("Tokenized Sentence:",tokens)
print("Filterd Sentence:",filtered_text)
```

Step 5 : Perform Stemming

```
from nltk.stem import PorterStemmer
e_words= ["wait", "waiting", "waited", "waits"]
ps =PorterStemmer()
for w in e_words:
    rootWord=ps.stem(w)
    print(rootWord)
```

Step 6: Perform Lemmatization

```
from nltk.stem import WordNetLemmatizer
wordnet_lemmatizer = WordNetLemmatizer()
text = "studies studying cries cry"
tokenization = nltk.word_tokenize(text)
for w in tokenization:
    print("Lemma for {} is {}".format(w,
    wordnet_lemmatizer.lemmatize(w)))
```

Step 7: Apply POS Tagging to text

```
import nltk
from nltk.tokenize import word_tokenize
data="The pink sweater fit her perfectly"
words=word_tokenize(data)
for word in words:
    print(nltk.pos_tag([word]))
```

Algorithm for Create representation of document by calculating TFIDF**Step 1: Import the necessary libraries.**

```
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
```

Step 2: Initialize the Documents.

```
documentA = 'Jupiter is the largest Planet'
documentB = 'Mars is the fourth planet from the Sun'
```

Step 3: Create BagofWords (BoW) for Document A and B.

```
bagOfWordsA = documentA.split(' ')
bagOfWordsB = documentB.split(' ')
```

Step 4: Create Collection of Unique words from Document A and B.

```
uniqueWords = set(bagOfWordsA).union(set(bagOfWordsB))
```

Step 5: Create a dictionary of words and their occurrence for each document in the corpus

```
numOfWordsA = dict.fromkeys(uniqueWords, 0)
for word in bagOfWordsA:
    numOfWordsA[word] += 1
numOfWordsB = dict.fromkeys(uniqueWords, 0)
for word in bagOfWordsB:
    numOfWordsB[word] += 1
```

Step 6: Compute the term frequency for each of our documents.

```
def computeTF(wordDict, bagOfWords):
    tfDict = {}
    bagOfWordsCount = len(bagOfWords)
    for word, count in wordDict.items():
        tfDict[word] = count / float(bagOfWordsCount)
    return tfDict
tfA = computeTF(numOfWordsA, bagOfWordsA)
tfB = computeTF(numOfWordsB, bagOfWordsB)
```

Step 7: Compute the term Inverse Document Frequency.

```
def computeIDF(documents):
    import math
    N = len(documents)

    idfDict = dict.fromkeys(documents[0].keys(), 0)
    for document in documents:
        for word, val in document.items():
            if val > 0:
                idfDict[word] += 1

    for word, val in idfDict.items():
        idfDict[word] = math.log(N / float(val))
    return idfDict
idfs = computeIDF([numOfWordsA, numOfWordsB])
idfs
```

Step 8: Compute the term TF/IDF for all words.

```
def computeTFIDF(tfBagOfWords, idfs):
    tfidf = {}
    for word, val in tfBagOfWords.items():
        tfidf[word] = val * idfs[word]
    return tfidf
```



```
tfidfA = computeTFIDF(tfA, idfs)
tfidfB = computeTFIDF(tfB, idfs)
df = pd.DataFrame([tfidfA, tfidfB])
df
```

Conclusion:

In this way we have done text data analysis using TF IDF algorithm

Assignment Question:

- 1) **Perform Stemming for** `text = "studies studying cries cry"`. **Compare the results generated with Lemmatization. Comment on your answer how Stemming and Lemmatization differ from each other.**
- 2) **Write Python code for removing stop words from the below documents, convert the documents into lowercase and calculate the TF, IDF and TFIDF score for each document.**

```
documentA = 'Jupiter is the largest Planet'
documentB = 'Mars is the fourth planet from the Sun'
```

Assignment No 8:

Title: Data Visualization I

1. Use the inbuilt dataset 'titanic'. The dataset contains 891 rows and contains information about the passengers who boarded the unfortunate Titanic ship. Use the Seaborn library to see if we can find any patterns in the data.
2. Write a code to check how the price of the ticket (column name: 'fare') for each passenger is distributed by plotting a histogram.

Theory: -

Data visualization is the graphical representation of information and data. By using visual elements like charts, graphs, and maps, data visualization tools provide an accessible way to see and understand trends, outliers, and patterns in data. In the world of Big Data, data visualization tools and technologies are essential to analyze massive amounts of information and make data-driven decisions

The advantages and benefits of good data visualization:

Data visualization is another form of visual art that grabs our interest and keeps our eyes on the message. When we see a chart, we quickly see trends and outliers. If we can see something, we internalize it quickly.

Patterns of data can be find out with the help of different types of plots

Types of plots are:

A. Distribution Plots

- a. Dist-Plot
- b. Joint Plot
- d. Rug Plot

B. Categorical Plots

- a. Bar Plot
- b. Count Plot
- c. Box Plot
- d. Violin Plot

C. Advanced Plots

- a. Strip Plot
- b. Swarm Plot

D. Matrix Plots

a. Heat Map

b. Cluster Map

A. Distribution Plots:

These plots help us to visualize the distribution of data. We can use these plots to understand the mean, median, range, variance, deviation, etc of the data.

a. Distplot

- Dist plot gives us the histogram of the selected continuous variable.
- It is an example of a univariate analysis.
- We can change the number of bins i.e. number of vertical bars in a histogram

b. Joint Plot

- It is the combination of the distplot of two variables.
- It is an example of bivariate analysis.
- We additionally obtain a scatter plot between the variables to reflect their linear relationship. We can customise the scatter plot into a hexagonal plot, where, the more the colour intensity, the more will be the number of observations.

c. The Rug Plot

The rugplot() is used to draw small bars along the x-axis for each point in the dataset. To plot a rug plot, you need to pass the name of the column. Let's plot a rug plot for fare.

Algorithm:

Step 1: Download the data set of Titanic (<https://www.kaggle.com/uciml/iris>)

Step 2: Importing Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
dataset = sns.load_dataset('titanic')
dataset.head()
```

Step 3: Draw distributional plot

```
sns.distplot(dataset['fare'])
```

Step 4: Removal of Kernal Density line

```
sns.distplot(dataset['fare'], kde=False)
```

Step 5: Draw histogram

```
sns.histplot(data=dataset, x="fare", binwidth=30)
```

Step 6: The Joint Plot

```
sns.jointplot(x='age', y='fare', data=dataset)
```

Step 7: The Pair Plot

```
sns.pairplot(dataset)
```

Question: Draw Histogram, Joint Plot.

Conclusion: Implemented successfully Simple Data visualization techniques using Python on Titanic dataset.

Assignment No 9:

Title: Data Visualization II

1. Use the inbuilt dataset 'titanic'. Plot a box plot for distribution of age with respect to each gender along with the information about whether they survived or not. (Column names : 'sex' and 'age')
2. Write observations on the inference from the above statistics.

Theory:

The Bar Plot

The barplot() is used to display the mean value for each value in a categorical column, against a numeric column. The first parameter is the categorical column, the second parameter is the numeric column while the third parameter is the dataset.

Algorithm:

Step 1: Download the data set of Titanic

Step 2: Importing Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Step 3: Reading of dataset

```
df = pd.read_csv('titanic')
```

Step 4: Drop columns

```
df_cleaned=df.drop(['pclass','embarked','deck','embark_town'],axis=1)
```

Step 5: Draw Box Plot

```
sns.boxplot(data=df,x="sex",y="age", color="Purple")
sns.boxplot(data=df,x="sex",y="age", hue="survived")
```

Question: Write the observations from the box plot.

Conclusion: Implemented successfully Box Plot Data visualization techniques using Python on Titanic dataset.

Assignment No 10:

Title: Data Visualization III

Download the Iris flower dataset or any other dataset into a DataFrame.
(e.g., <https://archive.ics.uci.edu/ml/datasets/Iris>).

Scan the dataset and give the inference as:

1. List down the features and their types (e.g., numeric, nominal) available in the dataset.
2. Create a histogram for each feature in the dataset to illustrate the feature distributions.
3. Create a box plot for each feature in the dataset.
4. Compare distributions and identify outliers.

Theory:

Features of a Dataset

The features of a dataset may allude to the columns available in the dataset. The features of a dataset are the most critical aspect of the dataset, as based on the features of each available data point, will there be any possibility of deploying models to find the output to predict the features of any new data point that may be added to the dataset.

Some possible features of a dataset are:

Numerical Features: These may include numerical values such as height, weight, and so on. These may be continuous over an interval, or discrete variables.

Categorical Features: These include multiple classes/ categories, such as gender, colour, and so on.

Metadata: Includes a general description of a dataset. Generally in very large datasets, having an idea/ description of the dataset when it's transferred to a new developer will save a lot of time and improve efficiency.

Size of the Data: It refers to the number of entries and features it contains in the file containing the Dataset.

Formatting of Data: The datasets available online are available in several formats. Some of them are JSON (JavaScript Object Notation), CSV (Comma Separated Value), XML (eXtensible Markup Language), DataFrame, and Excel Files (xlsx or xlsxm). For particularly large datasets, especially involving images for disease detection, while downloading the files from the internet, it comes in zip files which will be needed to extract in the system to individual components.

Algorithm:

Step 1: Download the data set of Iris

Step 2: Importing Libraries

```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
df1=pd.read_csv("Iris_data_sample.csv")
df1.head()
```

Step 3: Identify Features & data types

```
df1.shape
df1.info()
df1["Species"].unique()
df1.groupby("Species").size()
corr=df1.corr()
plt.subplots(figsize=(6,6))
sns.heatmap(corr, annot=True)
```

Step 4: Draw Box Plot

```
def graph(y):
sns.boxplot(x="Species", y=y, data=df1, color="purple")
plt.figure(figsize=(20,20))
```

Step 5: Adding the subplot at the specified grid position

```
plt.subplot(221)
graph('SepalLengthCm')
plt.subplot(222)
graph('SepalWidthCm')
plt.subplot(223)
graph('PetalLengthCm')
plt.show()
```

Questions: Identify the outliers from the boxplot drawn for iris dataset.

Conclusion: Implemented successfully data visualization on dataset features using Python on Iris dataset.